

```
!pip install boto3
```

```
Collecting boto3
  Downloading boto3-1.40.41-py3-none-any.whl.metadata (6.7 kB)
Collecting botocore<1.41.0,>=1.40.41 (from boto3)
  Downloading botocore-1.40.41-py3-none-any.whl.metadata (5.7 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3)
  Downloading jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.15.0,>=0.14.0 (from boto3)
  Downloading s3transfer-0.14.0-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3.0.0,>=2.1->botocore<1.41.0,>=1.40.41)
Downloading boto3-1.40.41-py3-none-any.whl (139 kB)
  139.3/139.3 kB 3.0 MB/s eta 0:00:00
Downloading botocore-1.40.41-py3-none-any.whl (14.0 MB)
  14.0/14.0 MB 97.3 MB/s eta 0:00:00
Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Downloading s3transfer-0.14.0-py3-none-any.whl (85 kB)
  85.7/85.7 kB 5.9 MB/s eta 0:00:00
Installing collected packages: jmespath, botocore, s3transfer, boto3
Successfully installed boto3-1.40.41 botocore-1.40.41 jmespath-1.0.1 s3transfer-0.14.0
```

```
#!/usr/bin/env python3
```

```
"""
```

```
prompt_variation_pipeline.py
```

```
Run multiple prompt variations on a task using AWS Bedrock (Claude 3 Sonnet)
without Docker or aws configure. Performs basic syntax checks.
```

```
"""
```

```
import boto3
```

```
import json
```

```
# ----- AWS Bedrock direct credentials -----
```

```
AWS_ACCESS_KEY = "AKIAVSX4Z23PH4R4CRWQ"
```

```
AWS_SECRET_KEY = "kIu79SSQ/aw0RL/cTFbhk20cjxjvzLhP8iEzXtmx"
```

```
AWS_REGION = "us-east-1" # region where Claude 3 Sonnet is available
```

```
MODEL_ID = "anthropic.claude-3-sonnet-20240229-v1:0"
```

```
# ----- Prompt variations -----
```

```
prompt_variations = [
```

```
{
```

```
    "name": "concise_zero_shot",
```

```
    "prompt": "Write Python function factorial(n:int)->int. Raise ValueError if n<0.",
```

```
    "temperature": 0
```

```
},
```

```
{
```

```
    "name": "detailed_zero_shot",
```

```
    "prompt": "You are a Python developer. Implement factorial(n:int)->int. Include docstring, type hints, and minimal comments.",
```

```
    "temperature": 0
```

```
},
```

```
{
```

```
    "name": "few_shot",
```

```
    "prompt": """You are a professional Python developer. Here is an example:
```

```
Spec: add(a:int, b:int) -> int that returns a+b
```

```
Code:
```

```
def add(a:int,b:int)->int:
```

```
    \"\"\"Return sum of a and b\"\"\"
```

```
    return a + b
```

```
Now implement factorial(n:int)->int. Raise ValueError if n<0.
```

```
\"\"\",
```

```
    "temperature": 0
```

```
    }
```

```
]

# ----- Function to invoke Bedrock model -----
```

```
def invoke_model(prompt: str, temperature: float = 0.2) -> str:
```

```
    rt = boto3.client(
```

```
        "bedrock-runtime",
```

```
        region_name=AWS_REGION,
```

```
        aws_access_key_id=AWS_ACCESS_KEY,
```

```
        aws_secret_access_key=AWS_SECRET_KEY
```

```
    )
```

```
    body = {
```

```
        "anthropic_version": "bedrock-2023-05-31",
```

```
        "max_tokens": 500,
```

```
        "temperature": temperature,
```

```
        "messages": [{"role": "user", "content": prompt}
    ]
```

```
    }
```

```
resp = rt.invoke_model(
    modelId=MODEL_ID,
    body=json.dumps(body),
    contentType="application/json"
)
obj = json.loads(resp["body"].read())
if "content" in obj and len(obj["content"]) > 0:
    return obj["content"][0].get("text", "").strip()
return ""

# ----- Basic syntax check -----
def check_syntax(code_text: str) -> bool:
    """
    Returns True if the code compiles (basic Python syntax check), else False
    """
    try:
        compile(code_text, "<string>", "exec")
        return True
    except Exception as e:
        print(f"Syntax Error: {e}")
        return False

# ----- Run prompt variations -----
def run_variations():
    results = []
    for var in prompt_variations:
        print(f"\n=== Running Variation: {var['name']} ===")
        output = invoke_model(var['prompt'], temperature=var['temperature'])
        # remove ``` if model returned code block
        if output.startswith("```"):
            output = output.strip("```").strip()
        print("Generated Code:\n", output)
        syntax_ok = check_syntax(output)
        print("Syntax check:", "PASS" if syntax_ok else "FAIL")
        results.append({
            "variation": var['name'],
            "prompt": var['prompt'],
            "output": output,
            "syntax_ok": syntax_ok
        })
    return results

# ----- Main -----
if __name__ == "__main__":
    run_variations()
```

2. The docstring explains the purpose of the function, the expected input, the output, and the possible exception that can be
3. The first `if` statement checks if `n` is negative. If it is, a `ValueError` is raised with an appropriate error message.
4. If `n` is non-negative, the function initializes a variable `result` to 1.
5. The `for` loop iterates from 1 to `n` (inclusive), and at each iteration, it multiplies the current value of `result` by `i`.
6. After the loop completes, `result` contains the factorial of `n`.
7. The function returns the value of `result`.

Here are some examples of how to use the `factorial` function:

```
```python
print(factorial(5)) # Output: 120
print(factorial(0)) # Output: 1
print(factorial(-3)) # Raises ValueError: n must be a non-negative integer
```
```

Note that the factorial function is defined using type hints for the input parameter `n` and the return value. This helps catch syntax errors like the one below.

```
SyntaxError: unterminated string literal (detected at line 1) (<string>, line 1)
```